

Học cách lấy mẫu đồ thị cho hệ gợi ý quy mô lớn dùng GNN

GRAPES-GFN-Rec trên Amazon Reviews'23 Baby_Products

Luận văn — bản số 0.3

Ty Van Xuan

Cập nhật 16/09/2026

Tóm tắt nội dung

Luận văn chuyển GRAPES [1] — phương pháp lấy mẫu đồ thị trong đó một GNN phụ học chọn node cho từng layer bằng GFlowNet — sang hệ gợi ý. Phương pháp đề xuất, GRAPES-GFN-Rec, học chọn node ngữ cảnh cho Sampled LightGCN [2] với phần thưởng từ BPR ranking loss [4] và Trajectory Balance [3], rồi được so sánh với lấy mẫu đều, lấy mẫu theo bậc, biến thể REINFORCE, Full LightGCN, MostPop và BPR-MF dưới cùng protocol. Dữ liệu là Amazon Reviews’23 Baby_Products [10]: graph huấn luyện 3.868.654 cạnh, 2.318.308 user, 162.125 sản phẩm, rất thưa (71,76% user một tương tác) và lệch long-tail (Gini 0,858). Đây là bản sống: chương dữ liệu, trả lời góp ý và phương pháp đã có; kết quả thực nghiệm được điền theo từng gate.

Mục lục

Trạng thái tài liệu và nhật ký cập nhật	3
1 Mở đầu	4
1.1 GNN cho hệ gợi ý và bùng nổ lân cận	4
1.2 Sampling hay distributed?	4
1.3 Giai đoạn 1: GRAPES	4
1.4 Rà soát hướng đi	5
1.5 Câu hỏi nghiên cứu	5
2 Dữ liệu và phân tích	6
2.1 Chọn dataset	6
2.2 Biểu diễn dữ liệu	6
2.3 Chất lượng dữ liệu và nhiễu	6
2.4 Rating và chọn tương tác positive	7
2.5 Bao nhiêu sản phẩm và histogram theo sản phẩm	7
2.6 Mất cân bằng và long-tail	8
2.7 Không đủ dữ liệu: cold-start	8
2.8 Dữ liệu theo thời gian	8
2.9 Cấu trúc graph huấn luyện	9
2.10 Metadata sản phẩm	9
2.11 Đường đi dữ liệu và chia split	9
2.12 Từ dữ liệu đến quyết định thiết kế	10
3 Trả lời góp ý của giảng viên (3/9)	11
3.1 “Converat” — coverage và conversion	11
3.2 Context	11
3.3 NDCG	11
3.4 Mức khớp ngữ nghĩa và đa dạng sản phẩm được gợi ý	11
3.5 Tính phân bố	11
3.6 Trade-off và “quan tâm tiêu chí nào thì giá trị khác ra sao”	12
3.7 Phân tích dữ liệu ban đầu	12
3.8 Sampling hay distributed	12
3.9 Long-tail và quan hệ giữa các node có giá trị ra sao	12
4 Phương pháp GRAPES-GFN-Rec	13
4.1 Kiến trúc tổng thể	13
4.2 Chuyển đổi từ GRAPES sang gợi ý	13
4.3 Lấy mẫu theo layer	13
4.4 Sampled LightGCN	14
4.5 Hàm mục tiêu	14
4.6 Những điểm riêng của bài toán gợi ý	14
5 Thiết kế thực nghiệm	16
5.1 Phương pháp so sánh	16
5.2 Chỉ số	16
5.3 Tiêu chí kết luận	16

6	Kết quả	17
6.1	Tiến độ đã kiểm chứng	17
6.2	Development (R3)	17
6.3	So sánh ghép cặp trên holdout (R5)	17
6.4	Hành vi sampler và độ nhạy budget (R6)	17
7	Kết luận và hướng phát triển	18
A	Kế hoạch đến hạn nộp 30/11/2026	19
B	Pilot M0–M2 (lưu trữ)	20

Trạng thái tài liệu và nhật ký cập nhật

Đây là **bản sống** của luận văn: mỗi gate hoàn thành sẽ điền thêm vào đúng chương, không viết lại từ đầu. Phần đóng khung “ĐANG CHỜ” chưa có số liệu và ghi rõ nguồn sẽ điền.

Chương	Nội dung	Trạng thái	Điền tiếp khi
1 Mở đầu	Bùng nổ lân cận, sampling vs distributed, GRAPES, câu hỏi nghiên cứu	Có	–
2 Dữ liệu	Chọn dataset, chất lượng, rating, histogram, long-tail, cold-start, thời gian, graph	Một phần	Notebook 11
3 Góp ý giảng viên	Trả lời từng câu hỏi ngày 3/9	Có; số ngữ nghĩa chờ	Notebook 11, R5
4 Phương pháp	GRAPES-GFN-Rec	Có	R2 hoàn tất
5 Thiết kế thực nghiệm	So sánh, chỉ số, tiêu chí	Có	R4
6 Kết quả	Development, holdout, hành vi sampler, budget	Đang chờ	R3, R5, R6
7 Kết luận	Tổng kết, hạn chế, hướng phát triển	Đang chờ	R7

Nhật ký.

- **0.3 (16/09/2026)** Chuyển sang bản sống; mở rộng chương dữ liệu; thêm chương trả lời góp ý 3/9; kế hoạch nén về hạn nộp 30/11/2026.
- **0.2 (16/09/2026)** Đề xuất GRAPES-GFN-Rec; lỗi mã R2 với 28 oracle test.
- **0.1 (16/09/2026)** Rà soát scope (DL-001): pilot M0–M2 chuyển sang phụ lục.

1 Mở đầu

1.1 GNN cho hệ gợi ý và bùng nổ lân cận

Trong collaborative filtering dựa trên đồ thị, user và sản phẩm là node, mỗi tương tác là một cạnh. LightGCN [2] học embedding bằng cách lan truyền và lấy trung bình embedding của hàng xóm qua L bước, không có ma trận biến đổi hay hàm phi tuyến. Để tính embedding của một user ở độ sâu L , model cần toàn bộ vùng lân cận L bước.

Kích thước vùng lân cận tăng theo tích các bậc gặp trên đường đi. Trên graph Baby_Products, một user trung bình có 1,67 sản phẩm; nhưng đi theo một cạnh ngẫu nhiên, sản phẩm gặp phải có bậc kỳ vọng

$$\mathbb{E}_{\text{cạnh}}[\deg(i)] = \frac{\sum_i \deg(i)^2}{\sum_i \deg(i)} \approx 1.009 \text{ user},$$

lớn hơn rất nhiều so với bậc trung bình 23,9, vì cạnh dồn vào sản phẩm phổ biến. Sản phẩm lớn nhất có 21.348 user. Chỉ sau hai bước, một user có thể kéo theo hàng nghìn user khác; một batch nhiều triplet hợp các vùng này lại. Đây là *bùng nổ lân cận* mà GraphSAGE [8], FastGCN [6], LADIES [7], PinSage [9] tìm cách kiểm soát.

ĐANG CHỜ · Notebook 11 (S4)

Kích thước lân cận 1–3 bước thực tế của user và tỷ lệ graph bị chậm theo batch 1.024–65.536 triplet.

1.2 Sampling hay distributed?

Có năm hướng chính để huấn luyện GNN trên graph lớn (Bảng 1.1).

Bảng 1.1: Các hướng mở rộng huấn luyện GNN.

Hướng	Cách làm	Hạn chế với bài toán này	Vai trò
Full-graph, GPU	1 Lan truyền trên toàn graph	Bộ nhớ và thời gian tăng theo số cạnh $\times$ số layer	Tham chiếu
Distributed training	Chia batch/model cho nhiều GPU, nhiều máy (PinSage [9])	Cần cụm máy, chi phí giao tiếp; không giảm lượng tính mỗi batch	Ngoài phạm vi
Chia đồ thị	Cluster-GCN [11]: học trên cụm con	Cắt cạnh giữa cụm; graph lệch hub khó chia đều	Không chọn
Embedding lịch sử	GAS [12]: dùng lại embedding cũ của hàng xóm	Lưu embedding 2,48 triệu node mỗi layer; embedding cũ bị lệch	Không chọn
Sampling	GraphSAGE, FastGCN, LADIES, GRAPES: giới hạn k node mỗi layer	Mất ngữ cảnh $\rightarrow$ phải chọn node đúng	Hướng chính

Hai hướng bổ sung cho nhau: distributed nhân phần cứng, sampling giảm lượng tính mỗi batch; sampler học được có thể chạy trong từng worker của hệ distributed. Đề án chọn sampling vì đề tài hướng tới và giới hạn một GPU T4.

1.3 Giai đoạn 1: GRAPES

GRAPES [1] dùng hai GNN: GCN_C giải bài toán chính và GCN_S chấm điểm hàng xóm để chọn đúng k node mỗi layer bằng Gumbel Top- k [5]; GCN_S được huấn luyện bằng GFlowNet Trajectory Balance [3] hoặc REINFORCE với chi phí là loss của GCN_C .

Bài học: lợi thế của sampler học được nhỏ và không đều; sampler tốn thêm 37–85% thời gian, nên chi phí sampler phải nằm trong so sánh. GRAPES chưa được đánh giá trên recommendation.

Bảng 1.2: Tái hiện GRAPES ở giai đoạn 1 trên Tesla T4 (accuracy).

Dataset	GRAPES	Random	Thời gian GRAPES	Thời gian Random
Cora	0,8710	0,8677	56,3 s	30,5 s
Citeseer	0,7857	0,7900	94,6 s	56,0 s
ogbn-arxiv	0,6204	0,6128	11.114 s	8.133 s

1.4 Rà soát hướng đi

Nhánh pilot trước đó so sánh ba luật chọn node cố định (M0 đều, M1 theo bậc, M2 frontier-normalized). M2 mượn hình thức hành động của GRAPES nhưng không có policy học được, log-likelihood, phần thưởng, mục tiêu TB/REINFORCE hay bước cập nhật sampler. Decision log DL-001 xác định lại: giai đoạn 2 hiện thực biến thể GRAPES học được; pilot chuyển phụ lục. Một phương pháp chỉ là *biến thể GRAPES* khi thỏa G1–G7: policy có tham số; exact- k Gumbel Top- k ; log q ; tín hiệu loss gợi ý đã detach; log $Z(V^0)$ học được; TB/REINFORCE; tham số sampler thực sự thay đổi.

1.5 Câu hỏi nghiên cứu

RQ1 Giữ cố định recommender, graph, thứ tự triplet, negative, số bước, budget, seed, evaluator và phần cứng, GRAPES-GFN-Rec có trade-off NDCG@20–chi phí tốt hơn lấy mẫu đều và theo bậc không?

RQ2 Trajectory Balance có ổn định, hiệu quả hơn REINFORCE trên tín hiệu BPR không?

RQ3 Sampler học được chọn node nào (bậc, loại, head/body/tail)? Có khuếch đại thiên lệch phổ biến không?

RQ4 Khi budget k chặt hơn, lợi thế (nếu có) thay đổi thế nào?

Kết quả âm là kết quả hợp lệ.

2 Dữ liệu và phân tích

2.1 Chọn dataset

Amazon Reviews’23 [10] có nhiều danh mục, số giao dịch lớn và metadata sản phẩm. Ba danh mục được kiểm toán chính xác trên file tải về (Bảng 2.1).

Bảng 2.1: So sánh ba danh mục đã kiểm toán (P4, cùng mốc t_1, t_2).

	All_Beauty	Baby_Products	Home_and_Kitchen
Dòng raw	693.929	5.953.891	66.623.880
Tương tác P4	494.769	4.655.843	chưa kiểm toán
User P4	455.579	2.769.312	–
Sản phẩm P4	91.187	194.722	–
User chỉ 1 tương tác	94,16%	71,64%	–
Target validation warm	1.802 (3,98%)	81.871 (21,90%)	–

All_Beauty quá thừa để đánh giá (94% user một tương tác, chỉ 1.802 target warm). Home_and_Kitchen lớn hơn Baby 11,2 lần, dành cho stress test sau. Baby_Products đủ lớn để thấy chi phí lấy mẫu, có long-tail rõ và vẫn chạy được ma trận thí nghiệm trên Colab T4. MovieLens 25M, Gowalla, Yelp2018, MIND, KuaiRec phù hợp câu hỏi khác nên chỉ dùng làm bối cảnh.

2.2 Biểu diễn dữ liệu

File gốc có bốn cột: `user_id`, `parent_asin`, `rating`, `timestamp` (ms). Dữ liệu được biểu diễn thành graph hai phía $G = (U \cup I, E)$: node là user hoặc sản phẩm, cạnh là tương tác 4–5★ kèm thời điểm. Rating không làm trọng số cạnh; không có feature node, nên mỗi node học một embedding ID. Metadata sản phẩm (title, danh mục, giá) chỉ dùng cho đánh giá ngữ nghĩa, không đưa vào huấn luyện.

2.3 Chất lượng dữ liệu và nhiễu

Bảng 2.2: Các kiểu nhiễu và quyết định.

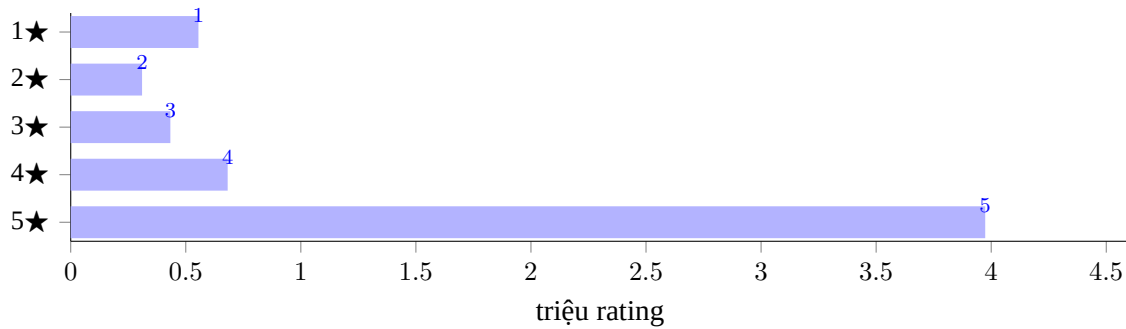
Loại	Mức độ	Quyết định	Lý do
Rating ngoài 1–5	1 dòng (0.0)	Loại	Lỗi rõ ràng
Trùng cặp, thiếu ID, sai timestamp	0	–	Kiểm tra chính xác toàn file
Rating 1–3★	1.298.047 (21,8%)	Không làm positive	Tín hiệu không thích hoặc không chắc, không phải lỗi
User/sản phẩm 1 tương tác	71,76% user; 33,35% sản phẩm	Giữ, không lọc k -core	Long-tail thật; lọc sẽ xóa đúng phần sampling phải xử lý
Bot, spam, đánh giá dồn dập	Chưa kiểm tra	Chỉ báo cáo	Không xóa khi chưa có quy tắc đăng ký trước
Thiên lệch phổ biến	Top 1% giữ 44,1%	Giữ, báo theo nhóm	Làm lệch chỉ số tổng

Nhiều dạng lỗi rất ít (1/5.953.891 dòng). 32.050 dòng có timestamp trùng với dòng khác; không dòng nào đúng bằng t_1 hay t_2 . Thách thức chính không phải nhiễu mà là thừa và mất cân bằng.

ĐANG CHỜ · Notebook 11 (S3)

Số user đánh giá ≥ 10 , ≥ 20 , ≥ 50 sản phẩm trong một ngày và tỷ lệ tương tác của họ; nhóm cùng timestamp của một user; sản phẩm tăng đột biến theo ngày; user có ≥ 10 rating giống hệt nhau.

2.4 Rating và chọn tương tác positive



Hình 2.1: Phân bố rating Baby_Products. Rating trung bình 4,21.

Phân bố hình chữ J: 66,7% là 5★, 21,8% là 1–3★, trung bình 4,21, nên 4★ đã thấp hơn trung bình. Bảng 2.3 so sánh ba cách chọn positive.

Bảng 2.3: Độ nhạy theo chính sách positive.

	Tất cả rating	P4: 4–5★	P5: chỉ 5★
Tương tác	5.953.890	4.655.843	3.973.866
User	3.386.206	2.769.312	2.476.012
Sản phẩm	217.654	194.722	182.226
User 1 tương tác	70,01%	71,64%	73,18%
Sản phẩm 1 tương tác	31,65%	33,13%	34,34%
Target validation warm	24,50%	21,90%	20,34%
User validation chưa thấy	75,07%	77,43%	78,88%

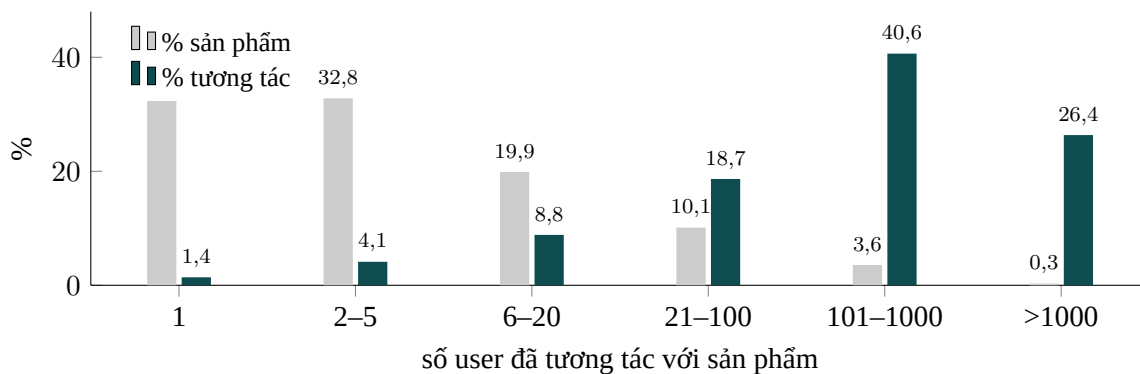
Coi mọi rating là positive sẽ gộp cả 1–3★ vào tín hiệu “thích”; chỉ lấy 5★ làm mất 14,65% tương tác và tăng cold-start. P4 là điểm cân bằng và được chọn trước khi có kết quả model.

ĐANG CHỜ · Notebook 11 (S1)

Rating trung bình, tỷ lệ 5★ và ≤ 3 ★ theo độ phổ biến sản phẩm và theo mức hoạt động của user; tỷ lệ user chỉ cho 5★; độ lệch rating mỗi user.

2.5 Bao nhiêu sản phẩm và histogram theo sản phẩm

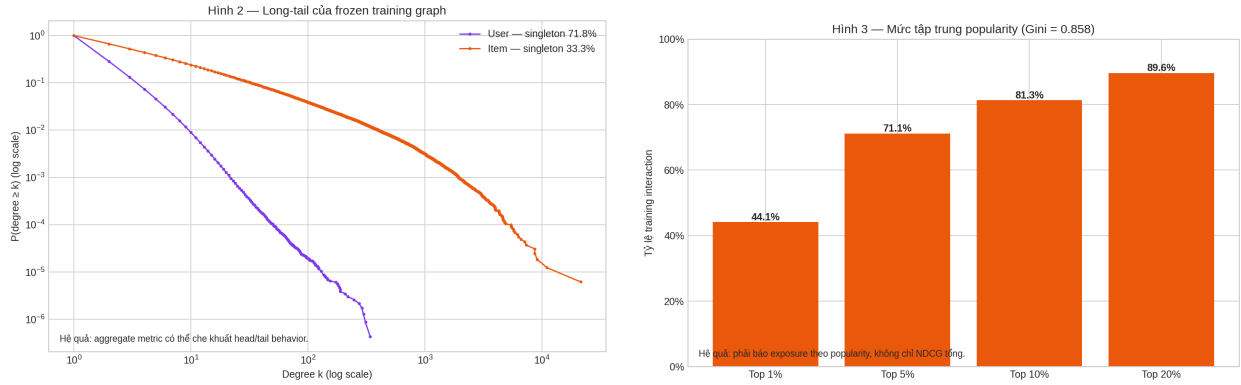
Số sản phẩm phụ thuộc bước lọc: 217.654 trong dữ liệu raw → 194.722 còn tương tác 4–5★ → 162.125 có trong graph train (trước t_1). Coverage dùng 162.125 làm mẫu số.



Hình 2.2: Histogram số tương tác của mỗi sản phẩm trong graph train.

33,35% sản phẩm chỉ có một tương tác nhưng chiếm 1,4% tương tác; 3,9% sản phẩm có trên 100 tương tác năm 67,0% tương tác. Phía user: 71,76% có 1 tương tác, 25,15% có 2–5, 2,97% có 6–20 và 0,11% có trên 20.

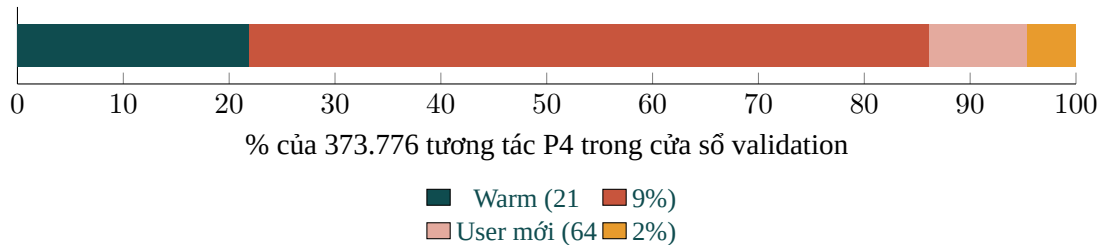
2.6 Mất cân bằng và long-tail



Hình 2.3: Trái: phân phối bậc (log–log). Phải: tỷ lệ tương tác do sản phẩm phổ biến nhất nắm giữ.

Gini bậc sản phẩm 0,8584; top 1% sản phẩm giữ 44,09% tương tác, top 20% giữ 89,65%. Nhóm theo bậc (khóa trước khi có kết quả model): head (≥ 397) có 1,00% sản phẩm và 44,16% cạnh; body (13–396) 18,92% và 45,45%; tail (≤ 12) 80,07% và 10,39%. Hệ quả: chỉ số tổng có thể tăng chỉ nhờ phục vụ tốt sản phẩm phổ biến; mọi kết quả phải kèm exposure và hit theo nhóm.

2.7 Không đủ dữ liệu: cold-start

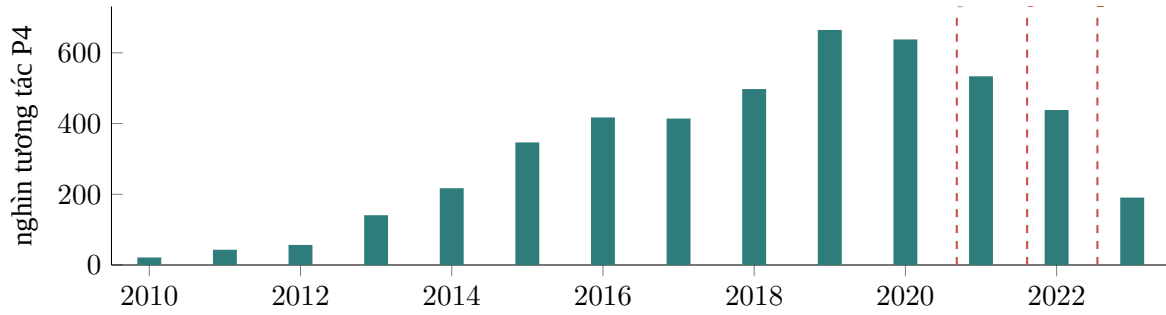


Hình 2.4: Thành phần tương tác tương lai so với graph train.

71,76% user train chỉ có một tương tác. Trong cửa sổ validation, 73,4% tương tác đến từ user chưa có trong train; chỉ 21,9% dùng được cho gợi ý warm-start, và con số này giảm còn 9,8% ở cửa sổ test. Đồ án giới hạn ở warm-start; cold-start cần feature nội dung và là hướng phát triển. Các số trên chỉ là số đếm dòng, không dùng target để thiết kế.

2.8 Dữ liệu theo thời gian

Dữ liệu trải từ 10/06/2000 đến 12/09/2023, tăng mạnh từ 2013, đỉnh 2019–2020 rồi giảm. Phân bố thay đổi theo thời gian nên đồ án chia theo thời gian thay vì ngẫu nhiên: $t_1 = 11/08/2021$, $t_2 = 16/07/2022$. Cửa sổ development $[t_0, t_1]$ với $t_0 = 05/09/2020$ ước tính khoảng 546 nghìn tương tác P4 (khoảng 14% trước t_1), nên quy tắc t_0 chính dự kiến đủ điều kiện.



Hình 2.5: Tương tác 4–5★ theo năm (2023 chỉ đến tháng 9).

ĐANG CHỜ · Notebook 11 (S2, R1)

User mới và sản phẩm mới theo năm; khoảng cách giữa hai tương tác của cùng user; thời gian sống của sản phẩm; độ trôi của top 1% sản phẩm giữa các năm; số liệu chính xác của development split.

2.9 Cấu trúc graph huấn luyện

Graph train có 2.480.433 node và 3.868.654 cạnh (mật độ $1,03 \times 10^{-5}$). 96,50% node thuộc thành phần liên thông lớn nhất (34.288 thành phần). Graph gần như liên thông nên lan truyền 2–3 bước chạm tới phần lớn graph; mỗi lựa chọn node vì vậy đánh đổi giữa tín hiệu phổ biến (dễ có, ít tính cá nhân) và tín hiệu hiếm.

ĐANG CHỜ · Notebook 11 (S4, R1)

Số cặp sản phẩm đồng mua; tỷ lệ sản phẩm không có sản phẩm đồng mua nào (không nhận được collaborative signal qua 2 bước); sản phẩm chỉ được user một-tương-tác chạm tới; target development nằm trong 3 bước của user bao nhiêu phần trăm, theo nhóm head/body/tail.

2.10 Metadata sản phẩm

Amazon Reviews’23 công bố meta_Baby_Products với title, categories (phân cấp), features, description, price, store, details [10]. Metadata cho phép đo mức khớp ngữ nghĩa và đa dạng (mục 3.4) mà không thay đổi phương pháp.

ĐANG CHỜ · Notebook 11 (S6)

Độ phủ title/danh mục/giá/store trên 162.125 sản phẩm train; độ sâu cây danh mục; top danh mục cấp 2 theo tỷ lệ sản phẩm và tỷ lệ tương tác; giá trung vị theo head/body/tail.

2.11 Đường đi dữ liệu và chia split

Raw CSV (kiểm SHA-256) → kiểm toán → P4 → chia theo thời gian → graph train và mapping ID → huấn luyện có lấy mẫu → đánh giá full-graph, toàn catalog.

Bảng 2.4: Vai trò các split.

Split	Dùng cho	Cấm
$G_{\text{dev_train}} (< t_0) / D_{\text{dev}} ([t_0, t_1))$	Chọn budget, α , learning rate, kiến trúc; kiểm tra backbone	Kết luận cuối
Graph train ($< t_1$) / validation	So sánh ghép cặp sau freeze	Mọi điều chỉnh
Test ($\geq t_2$)	Một lần, nếu có protocol riêng	Chọn phương pháp

Quy tắc dự phòng: nếu D_{dev} dưới 20.000 target hoặc $G_{\text{dev_train}}$ mất quá 50% cạnh, chọn t_0 sao cho $G_{\text{dev_train}}$ giữ 80% cạnh.

2.12 Từ dữ liệu đến quyết định thiết kế

Quan sát	Quyết định
Rating hình chữ J, 21,8% là 1–3★	Positive = 4–5★; không dùng rating làm trọng số
Nhiều dạng lỗi chỉ 1 dòng	Không lọc thêm; bất thường chỉ báo cáo
71,8% user và 33% sản phẩm có 1 tương tác	Không lọc k -core; theo dõi bậc của node được sampler chọn
Top 1% sản phẩm giữ 44% tương tác	Báo exposure/hit theo head–body–tail
73% tương tác validation của user mới	Giới hạn warm-start
Phân bố thay đổi mạnh theo năm	Chia theo thời gian; development split trước t_1
Graph gần liên thông, bậc kỳ vọng theo cạnh 1.009	Lấy mẫu có budget k ; tính chi phí sampler
Có metadata sản phẩm	Dùng cho đánh giá ngữ nghĩa, không huấn luyện

3 Trả lời góp ý của giảng viên (3/9)

Mỗi mục trình bày: câu hỏi, phân tích, số liệu và hệ quả cho thiết kế.

3.1 “Converat” — coverage và conversion

Coverage@20 là tỷ lệ sản phẩm trong catalog xuất hiện ít nhất một lần trong top-20 của mọi user; đo độ phủ, chưa đo ý nghĩa. **Conversion** là tỷ lệ gợi ý dẫn tới mua; cần log hiển thị–click–mua và A/B test online, dữ liệu review offline không có. Proxy offline gần nhất là Recall@20 trên tương tác tương lai. Ví dụ baseline sanity cũ: MostPop chỉ gợi ý 25 sản phẩm (coverage 0,00015) trong khi BPR-MF đạt 0,0335.

3.2 Context

Trong đồ án, *context* là tập node và cạnh lân cận model dùng để tính embedding cho batch ở mỗi layer (K^l). Sampler quyết định context. Đây không phải ngữ cảnh người dùng như thời gian, thiết bị, vị trí (context-aware recommendation, ngoài phạm vi).

3.3 NDCG

Mỗi dòng đánh giá có một sản phẩm đúng. Xếp hạng toàn bộ catalog hợp lệ, gọi r là hạng của sản phẩm đúng [14]:

$$\text{NDCG@20} = \begin{cases} 1/\log_2(r+1) & r \leq 20 \\ 0 & r > 20 \end{cases}$$

Hạng 1 cho 1; hạng 3 cho 0,5; hạng 10 cho 0,289; hạng 20 cho 0,228. NDCG thưởng việc đặt sản phẩm đúng lên cao; Recall@20 chỉ hỏi có vào top-20 hay không.

3.4 Mức khớp ngữ nghĩa và đa dạng sản phẩm được gợi ý

Với metadata, đồ án đề xuất ba phép đo bổ sung (cần giảng viên duyệt, OD-2), tính trên rank vector đã lưu nên không thay đổi phương pháp:

- **Semantic match@20**: $\max_{j \in \text{top-20}} \cos(e_j, e_{\text{đúng}})$, với e là embedding văn bản title + features [15].
- **ILD@20** [13]: $\frac{2}{k(k-1)} \sum_{i < j} d(i, j)$, với d là 1–Jaccard của đường danh mục hoặc $1 - \cos$ văn bản.
- **Đa dạng toàn hệ thống**: Coverage@20, Gini exposure, số danh mục xuất hiện trong gợi ý.

ĐANG CHỜ · Notebook 11 (S6), R5

Độ phủ metadata; giá trị semantic match@20 và ILD@20 cho từng phương pháp.

3.5 Tính phân bố

Exposure theo nhóm = tỷ lệ vị trí trong top-20 thuộc head/body/tail; hit theo nhóm = tỷ lệ target của nhóm được gợi ý đúng; Gini exposure đo mức dồn. Exposure tail tăng không đồng nghĩa hit tail tăng, nên báo cả hai.

Bảng 3.1: Nếu chỉ tối ưu một tiêu chí.

Tối ưu	Thường xảy ra	Bảng chứng trên Baby	Cách đo
NDCG tổng	Dồn về sản phẩm phổ biến	MostPop: NDCG@20 0,0059, 25 sản phẩm, 100% exposure head	NDCG + coverage + exposure
Độ phủ/đa dạng	Có thể giảm độ chính xác	BPR-MF: coverage 0,0335 ($\approx 218 \times$ MostPop), NDCG@20 0,0041	NDCG—coverage
Chi phí thấp (budget nhỏ)	Ít ngữ cảnh	Chưa đo (RQ4)	NDCG—thời gian
Đưa tail lên	Exposure tăng, hit chưa chắc	Chưa đo	Exposure và hit tail
Khớp ngữ nghĩa	Gợi ý giống nhau	Chưa đo	Semantic match + ILD

3.6 Trade-off và “quan tâm tiêu chí nào thì giá trị khác ra sao”

MostPop, BPR-MF là baseline sanity cũ (5 epoch, validation), chỉ minh họa trade-off. Kết quả cuối được trình bày như mặt trận Pareto: một phương pháp không được coi là tốt hơn nếu thua ở một trục mà không thắng ở trục khác.

3.7 Phân tích dữ liệu ban đầu

Các câu hỏi về số sản phẩm, histogram, mất cân bằng, dồn về, không đủ dữ liệu, rating, cách biểu diễn, nhiều và cân bằng được trả lời trong chương 2: mục 2.1–2.9 và Bảng 2.2, 2.3.

3.8 Sampling hay distributed

Xem mục 1.2: chọn sampling; distributed bổ sung, không thay thế.

3.9 Long-tail và quan hệ giữa các node có giá trị ra sao

- **user**→**sản phẩm**: lịch sử trực tiếp; với user một tương tác đây là toàn bộ thông tin.
- **sản phẩm**→**user**→**sản phẩm**: “người mua A cũng mua B”, collaborative signal cốt lõi, xuất hiện từ bước 2.
- **qua sản phẩm head**: nhiều tín hiệu phổ biến, ít tính cá nhân, tốn chi phí lan truyền.
- **qua sản phẩm tail**: tín hiệu riêng, ngách, ít quan sát nên dễ nhiễu.
- **qua user hoạt động nhiều**: cầu nối nhiều sản phẩm, có thể quá chung.

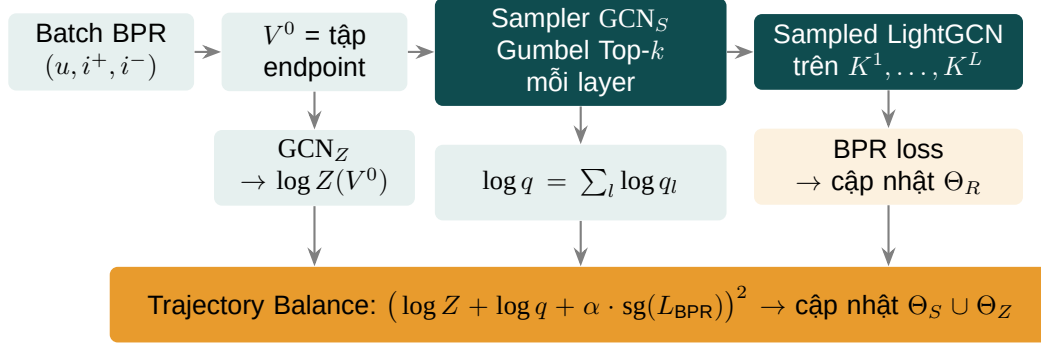
Heuristic cố định áp đặt trước giá trị (M1 ưu tiên head, M2 phạt hub). GRAPES-GFN-Rec để sampler học: quan hệ nào giúp BPR loss giảm sẽ được chọn nhiều hơn; RQ3 đo lại phân phối node được chọn và RQ4 cho biết quan hệ nào vẫn được giữ khi budget chặt.

ĐANG CHỜ · Notebook 11 (S4, R1), R6

Tỷ lệ sản phẩm không có đồng mua; target development trong 3 bước; phân phối node được sampler chọn.

4 Phương pháp GRAPES-GFN-Rec

4.1 Kiến trúc tổng thể



Hình 4.1: Một bước huấn luyện. sg là stop-gradient. Suy luận không lấy mẫu: LightGCN chạy trên toàn graph train, xếp hạng toàn catalog.

Hình 4.1 là khung kiến trúc chung. Sampler và recommender có tham số, optimizer và gradient tách biệt. Backbone có thể thay (ví dụ NGCF, SGL) mà không đổi sampler; trong triển khai thực tế, sampler chỉ dùng lúc huấn luyện, phục vụ gợi ý dùng embedding đã học.

4.2 Chuyển đổi từ GRAPES sang gợi ý

Bảng 4.1: Chuyển đổi thành phần. Các dòng bên phải là quyết định thiết kế của đồ án, không phải kết quả đã được paper GRAPES chứng minh.

Khía cạnh	GRAPES gốc	GRAPES-GFN-Rec
Đồ thị	Node có feature	Hai phía user-item, chỉ positive trước mốc thời gian
Mục tiêu batch	Node có nhãn	Triplet BPR; V^0 = unique endpoint, giữ ánh xạ triplet
Feature sampler	Feature node	Embedding ID riêng, one-hot loại node, $\log(1 + \text{deg})$ chuẩn hóa, cờ lịch sử $L+1$ kênh
Model chính	GCN phân loại	Sampled LightGCN: không self-loop, block $K^l \rightarrow K^{l-1}$, bi-normalization chữ nhật
Chi phí	Cross-entropy	Mean BPR loss, không gồm regularizer
$\log Z$	GCN trên target và hàng xóm	GCN _Z chỉ trên V^0
Đánh giá	Accuracy	Xếp hạng toàn catalog, NDCG@20, Recall@20, coverage, head/tail

4.3 Lấy mẫu theo layer

Với mỗi layer $l = 1, \dots, L$, tập ứng viên là hàng xóm của trạng thái trước trừ chính nó:

$$C^l = \mathcal{N}_{G_{\text{train}}}(K^{l-1}) \setminus K^{l-1}, \quad K^0 = V^0.$$

GCN_S hai lớp chạy trên subgraph cảm sinh bởi $K^{l-1} \cup C^l$ và cho logit ℓ_v , xác suất $p_v = \sigma(\ell_v)$ cho mỗi $v \in C^l$. Hành động chọn đúng min($k_l, |C^l|$) node:

$$V^l = \text{TopK}_{v \in C^l}(\log p_v + g_v), \quad g_v \sim \text{Gumbel}(0, 1), \quad K^l = V^0 \cup V^l.$$

Log-likelihood dùng phân phối Bernoulli đầy đủ, gồm cả node được chọn ($m_v = 1$) và không được chọn:

$$\log q_l = \sum_{v \in C^l} [m_v \log p_v + (1 - m_v) \log(1 - p_v)], \quad \log q = \sum_{l=1}^L \log q_l.$$

Hành động lấy từ phân phối exact- k trong khi likelihood là Bernoulli không điều kiện: đây là sai lệch off-policy kế thừa từ GRAPES và sẽ được nêu rõ.

Hai đối chứng M0 và M1 dùng cùng đường code, chỉ khác điểm: M0 dùng điểm bằng nhau, M1 dùng $\log \deg(v)$; cả hai không có tham số và $\log q = 0$. Nhờ vậy khác biệt giữa các sampler chỉ nằm ở cách chấm điểm.

4.4 Sampled LightGCN

Block $P_l \in \mathbb{R}^{|K^{l-1}| \times |K^l|}$ chứa cạnh gốc giữa nguồn K^l và đích K^{l-1} , chuẩn hóa hai phía theo bậc cục bộ trong block:

$$P_l[t, s] = \frac{B_l[t, s]}{\sqrt{d_l^{\text{dst}}(t) d_l^{\text{src}}(s)}}.$$

Embedding ở độ sâu r của V^0 là tích tiền tố $e^r = P_1 P_2 \cdots P_r E[K^r]$, và biểu diễn cuối là trung bình đều $z = \frac{1}{L+1} \sum_{r=0}^L e^r$.

Giới hạn đã được chứng minh bằng test. Vì K^l không cộng dồn, khi V^0 chứa cả user lẫn item, một số node thuộc V^1 bị loại khỏi K^2 ; do đó kể cả khi lấy hết ứng viên, kết quả không bằng Full LightGCN. Với target chỉ gồm một user, hai cách cho kết quả bằng nhau. Đây là tính chất của cách định nghĩa trạng thái trong GRAPES, không phải lỗi cài đặt.

4.5 Hàm mục tiêu

Recommender tối ưu

$$L_{\text{BPR}} = -\frac{1}{M} \sum_{b=1}^M \log \sigma(z_{u_b}^\top z_{i_b^+} - z_{u_b}^\top z_{i_b^-}), \quad L_{\text{model}} = L_{\text{BPR}} + \lambda L_{\text{reg}}.$$

Sampler tối ưu Trajectory Balance [3] với phần thưởng $R = \exp(-\alpha L_{\text{BPR}})$:

$$L_{\text{TB}} = \left(\log Z(V^0) + \log q + \alpha \cdot \text{sg}(L_{\text{BPR}}) \right)^2.$$

$\log Z(V^0)$ là trung bình đầu ra của GCN_Z trên subgraph cảm sinh bởi V^0 (có self-loop), nên bất biến với hoán vị và với node ngoài V^0 . Biến thể GRAPES-RL-Rec thay L_{TB} bằng $\text{sg}(L_{\text{BPR}}) \cdot \log q$.

4.6 Những điểm riêng của bài toán gợi ý

R-1 Quá ít phần thưởng.

Pilot dùng batch 65.536 trong 5 epoch, khoảng 300 bước tối ưu: sampler chỉ nhận khoảng 300 phần thưởng vô hướng cho hàng trăm nghìn node mục tiêu mỗi bước. GRAPES gốc dùng batch 256–512. Batch và budget sẽ được chọn lại trên development split, chung cho mọi sampler.

R-2 Thang phần thưởng.

BPR khởi đầu khoảng 0,69 và chênh lệch giữa trajectory rất nhỏ; nếu α quá nhỏ, $\log q$ lấn át phần thưởng. GRAPES gốc chọn hệ số trong khoảng 10^3 đến 10^5 . α và $\log Z$ khởi đầu được quét trên D_{dev} .

Thuật toán 1: Một bước huấn luyện GRAPES-GFN-Rec

Input: batch triplet B , graph G_{train} , budget $(k_1, \dots, k_L)$, α

- 1 $V^0 \leftarrow \text{unique}(B)$; giữ ánh xạ triplet $\rightarrow$ chỉ số;
- 2 **for** $l \leftarrow 1$ **to** L **do**
- 3 $C^l \leftarrow \mathcal{N}(K^{l-1}) \setminus K^{l-1}$; $\ell \leftarrow \text{GCN}_S(K^{l-1} \cup C^l)$;
- 4 $V^l \leftarrow \text{GumbelTopK}(\log \sigma(\ell), k_l)$; $\log q \leftarrow \log q + \log q_l$; $K^l \leftarrow V^0 \cup V^l$;
- 5 $z \leftarrow \text{SampledLightGCN}(P_1, \dots, P_L)$; tính L_{model} ;
- 6 Cập nhật Θ_R bằng ∇L_{model} ; // sampler không nhận gradient này
- 7 $L_{\text{TB}} \leftarrow (\log Z(V^0) + \log q + \alpha \text{sg}(L_{\text{BPR}}))^2$;
- 8 Cập nhật $\Theta_S \cup \Theta_Z$ bằng ∇L_{TB} ; // recommender không nhận gradient này
- 9 Ghi log: $L_{\text{BPR}}, L_{\text{TB}}, \log q, \log Z, |C^l|, |V^l|$, bậc/loại/nhóm của V^l , thời gian, bộ nhớ;

R-3 Phần thưởng không dừng.

Khi recommender học thêm, cùng trajectory cho loss khác. Giữ như GRAPES và ghi log mức trôi.

R-4 Graph cực thưa.

Theo dõi phân phối bậc, loại và nhóm của node được chọn để phát hiện sampler dồn về hub.

R-5 Chi phí sampler.

Embedding cho 2,48 triệu node cộng GCN_S có thể đắt hơn LightGCN; chi phí báo cáo gồm cả sampler.

R-6 Backbone yếu.

Trong pilot, Full LightGCN (NDCG@20 0,0049) thua MostPop (0,0059). Full LightGCN phải vượt MostPop trên D_{dev} trước khi so sánh sampler có ý nghĩa.

R-7, R-8.

Item âm cũng là mục tiêu của sampler; cạnh positive của batch có thể rò qua propagation, được kiểm tra bằng ablation che cạnh tạm thời.

5 Thiết kế thực nghiệm

5.1 Phương pháp so sánh

Bảng 5.1: Các phương pháp so sánh.

Tầng	Phương pháp	Vai trò	Câu hỏi trả lời
A	GRAPES-GFN-Rec	Phương pháp đề xuất	—
A	M0 Uniform	Đối chứng trung lập	Học có hơn chọn ngẫu nhiên?
A	M1 Degree-importance	Đối chứng tĩnh (họ FastGCN/LADIES [6, 7])	Học có hơn ưu tiên node phổ biến?
A	GRAPES-RL-Rec	Ablation hàm mục tiêu	TB có cần hơn REINFORCE?
B	Full LightGCN	Không lấy mẫu	Lấy mẫu mất bao nhiêu chất lượng?
B	MostPop, BPR-MF	Sanity	Graph có giá trị?
C	GraphSAGE fan-out [8]	Tùy chọn	Layer-wise học được vs node-wise?

Tầng A dùng chung graph, thứ tự triplet, negative, khởi tạo, budget, seed, evaluator và GPU, và được chạy lại hoàn toàn; số liệu pilot không được dùng lại.

5.2 Chỉ số

- **Chất lượng:** NDCG@20 (chính), Recall@20; chênh lệch ghép cặp theo seed.
- **Phân bố và đa dạng:** Catalog Coverage@20; exposure và hit theo head/body/tail; Gini exposure.
- **Chi phí:** thời gian sampler, TB, propagation và epoch; peak GPU memory; số node/cạnh thực tế mỗi layer.
- **Hành vi sampler:** bậc, loại, nhóm của node được chọn; entropy; $p_{\min}, p_{\max}$; độ thay đổi tham số sampler.
- **Ngữ nghĩa và đa dạng:** semantic match@20, ILD@20 (mục 3.4), nếu OD-2 được duyệt. Conversion không đo được offline.

5.3 Tiêu chí kết luận

Chỉ kết luận GRAPES-GFN-Rec cải thiện trade-off khi chênh lệch NDCG@20 so với đối chứng tầng A tốt nhất dương ở mọi seed holdout và chi phí được báo đầy đủ. Một run không hợp lệ nếu vi phạm cách ly thời gian, exact- k , quy tắc ứng viên, hướng block và normalization, tính hữu hạn của loss, phân quyền gradient, khả năng lặp lại hoặc thiếu log tài nguyên. Không đổi phương pháp, α , budget hay seed sau khi đọc holdout.

6 Kết quả

6.1 Tiến độ đã kiểm chứng

- **R0**: spec chuẩn, decision log DL-001, checklist G1–G7, công cụ kiểm tra tính nhất quán.
- **R2 (lỗi)**: sampler (learned, uniform, degree), Sampled LightGCN, bước huấn luyện TB/REINFORCE. 28 oracle test pass trên PyTorch CPU: tập ứng viên khớp brute-force; exact- k ; lấy mẫu đều khi điểm bằng nhau; log q khớp tính tay; đường đi mẫu cho 1 (không chuẩn hóa), $1/2$ (bậc toàn graph), $1/\sqrt{2}$ (bậc cục bộ); TB khớp tính tay; log Z bất biến hoán vị; TB không tạo gradient cho recommender và ngược lại; tham số sampler thay đổi sau một bước; lặp lại xác định. Bốn lỗi cài cố ý đều bị phát hiện.

6.2 Development (R3)

ĐANG CHỜ · Notebook 12, dự kiến 23/09–10/10

Full LightGCN vs MostPop theo số epoch; budget chung (batch, k , số bước, thời gian, bộ nhớ); quét α , log Z khởi đầu, learning rate; bằng chứng sampler học (G7); chi phí sampler.

6.3 So sánh ghép cặp trên holdout (R5)

ĐANG CHỜ · Notebook 13, dự kiến 14/10–27/10

NDCG@20, Recall@20 của 7 phương pháp $\times$ 3 seed và chênh lệch ghép cặp (RQ1); coverage, exposure/hit theo nhóm, Gini; thời gian và bộ nhớ; TB vs REINFORCE (RQ2); semantic match@20, ILD@20.

6.4 Hành vi sampler và độ nhạy budget (R6)

ĐANG CHỜ · Dự kiến 28/10–03/11

Bậc, loại, nhóm của node được chọn theo layer so với M0/M1 (RQ3); mặt trận Pareto; budget chặt (RQ4); ablation che cạnh positive (D9) và log Z kiểu cũ (D6).

7 Kết luận và hướng phát triển

ĐANG CHỜ · R7, dự kiến 04/11–17/11

Trả lời RQ1–RQ4 từ bằng chứng holdout; hạn chế (warm-start, một dataset, off-policy mismatch, K^l không cộng dồn); hướng phát triển: cold-start với metadata, Home_and_Kitchen, distributed + sampler học được.

Hạn chế đã biết từ thiết kế: chỉ warm-start; một dataset chính; sai lệch off-policy giữa hành động exact- k và likelihood Bernoulli (kể thừa GRAPES); trạng thái $K^l = V^0 \cup V^l$ không tương đương Full LightGCN khi V^0 có cả user lẫn sản phẩm; chỉ đo huấn luyện có lấy mẫu, không đo suy luận có lấy mẫu.

A Kế hoạch đến hạn nộp 30/11/2026

Tuần	Gate	Việc	Trạng thái
16/9	R0, R2 lỗi	Scope, spec, code, oracle test	Xong
16–22/9	R1	Notebook 11: phân tích sâu + development split; hoàn tất R2	Đang làm
23/9–10/10	R3	Backbone, budget, quét α/lr , khả thi T4	
10–13/10	R4	Gặp giảng viên chốt OD-1...OD-4; freeze	
14–27/10	R5	7 phương pháp $\times$ 3 seed, chạy Colab song song	
28/10–3/11	R6	Phân tích RQ1–RQ4, ngữ nghĩa, Pareto	
4–17/11	R7	Hoàn thiện luận văn và slide (viết song song từ tuần đầu)	
18–30/11	Dự phòng	Sửa theo góp ý, nộp	

Quyết định cần giảng viên: OD-1 thêm GraphSAGE fan-out (mặc định không); OD-2 nối metadata để đo ngữ nghĩa và đa dạng (khuyến nghị có); OD-3 lưới budget (một chính, một chặt); OD-4 số seed (3).

B Pilot M0–M2 (lưu trữ)

Pilot so sánh ba luật chọn node cố định trên validation với batch 65.536, 5 epoch (≈ 300 bước). Trung bình ba seed NDCG@20: M0 0,005706; M1 0,005866; M2 0,005721. M2 là heuristic tĩnh, không phải biến thể GRAPES; kết quả không dùng để chọn cấu hình hay kết luận về GRAPES-GFN-Rec. Bài học: quá ít bước cho sampler học (R-1) và backbone còn yếu (Full LightGCN 0,004931 < MostPop 0,005873, R-6).

Tài liệu tham khảo

- [1] T. Younesian et al. *GRAPES: Learning to Sample Graphs for Scalable Graph Neural Networks*. arXiv:2310.03399v3. Mã nguồn: <https://github.com/dfdazac/grapes>, commit 71e3e3e.
- [2] X. He et al. *LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation*. SIGIR 2020. arXiv:2002.02126.
- [3] N. Malkin et al. *Trajectory Balance: Improved Credit Assignment in GFlowNets*. NeurIPS 2022. arXiv:2201.13259.
- [4] S. Rendle et al. *BPR: Bayesian Personalized Ranking from Implicit Feedback*. UAI 2009. arXiv:1205.2618.
- [5] W. Kool et al. *Stochastic Beams and Where to Find Them: The Gumbel-Top-k Trick for Sampling Sequences Without Replacement*. ICML 2019. arXiv:1903.06059.
- [6] J. Chen et al. *FastGCN: Fast Learning with Graph Convolutional Networks via Importance Sampling*. ICLR 2018. arXiv:1801.10247.
- [7] D. Zou et al. *Layer-Dependent Importance Sampling for Training Deep and Large Graph Convolutional Networks*. NeurIPS 2019. arXiv:1911.07323.
- [8] W. Hamilton et al. *Inductive Representation Learning on Large Graphs*. NeurIPS 2017. arXiv:1706.02216.
- [9] R. Ying et al. *Graph Convolutional Neural Networks for Web-Scale Recommender Systems*. KDD 2018. arXiv:1806.01973.
- [10] Y. Hou et al. *Bridging Language and Items for Retrieval and Recommendation*. arXiv:2403.03952. Dữ liệu: <https://amazon-reviews-2023.github.io/>.
- [11] W.-L. Chiang et al. *Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks*. KDD 2019. arXiv:1905.07953.
- [12] M. Fey et al. *GNNAutoScale: Scalable and Expressive Graph Neural Networks via Historical Embeddings*. ICML 2021. arXiv:2106.05609.
- [13] C.-N. Ziegler et al. *Improving Recommendation Lists Through Topic Diversification*. WWW 2005.
- [14] K. Järvelin, J. Kekäläinen. *Cumulated Gain-Based Evaluation of IR Techniques*. ACM TOIS 20(4), 2002.
- [15] N. Reimers, I. Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP 2019. arXiv:1908.10084.